



Aula 11 - ORM Parte 3: Relacionamentos com o Eloquent - Relações Polimórficas

IFSUL | CSTSI | Prof. Gill Velleda Gonzales

LICENÇA:

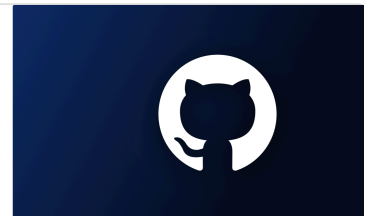
Creative Commons — Attribution-ShareAlike 4.0 International
— CC BY-SA 4.0

Nesta aula, finalizaremos a modelagem de relacionamentos com o ORM Eloquent com uma visão geral do suporte a relações polimórficas, uma técnica popularizada pelo Rails (Ruby). Veremos um exemplo aplicado no banco de dados de teste da disciplina. Ao final, será criada uma configuração inicial de logs do sistema para as consultas executadas no banco, além da instalação e uso da ferramenta de depuração Debugbar.

Repositório:

cstsi_daoo_20231_php_poo/aula_05 at main · g1ll/cstsi_daoo_20231_php_poo
CSTSI DAOO 2023/1: Exemplos de POO com PHP e Laravel -
cstsi_daoo_20231_php_poo/aula_05 at main · g1ll/cstsi_daoo_20231_php_poo

 https://github.com/g1ll/cstsi_daoo_20231_php_poo/tree/main/aula_05



Versões:

```
...correcoes/20231/aula_05 <?php 8.2.5 } correcao-aula-08-orm-eloquent-relationship-parte-2
$ php -v;composer -V; php artisan -V
PHP 8.2.5 (cli) (built: Apr 14 2023 04:26:42) (NTS)
Copyright (c) The PHP Group
Zend Engine v4.2.5, Copyright (c) Zend Technologies
    with Zend OPcache v8.2.5, Copyright (c), by Zend Technologies
Composer version 2.5.5 2023-03-21 11:50:05
Laravel Framework 10.5.1
```

Continuidade da [Aula 9](#).

Relações Polimórficas

As relações **Polimórficas** do Eloquent trazem a camada de ORM do sistema a ideia de polimorfismo do paradigma de **Programação Orientada a Objetos**. Nos exemplos a seguir, observamos que é possível criar um relacionamento com diferentes classes **Models** armazenando o tipo de modelo que realmente deverá ser instanciado no momento de carregar os objetos do relacionamento. Este tipo de relacionamento suporta todas as configurações já abordadas, **oneToOne**, **oneToMany** e **ManyToMany**:

OneToOne:

- **morphOne**: Aplicada nas classes Polimórficas, passando como parâmetro a model de destino da relação, similar ao **hasOne**.
- **morphTo**: Aplicada na classe Model de destino, similar ao **belongsTo**.

OneToMany:

- **morphMany**: Aplicada nas classes Polimórficas, passando como parâmetro a model de destino da relação, similar ao **hasMany**.
- **morphTo**: Aplicada na classe Model de destino, similar ao **belongsToMany**.

ManyToMany: utiliza o método **morphToMany**, similar ao **belongsToMany**.

Tabela Media

Para ilustrar melhor o funcionamento das relações polimórficas, aplicaremos no banco de dados de exemplo da disciplina uma relação polimórfica **oneToMany (morphMany)** entre as models Produto e Fornecedor com uma nova model chamada **Media**. A model **Media** deverá armazenar links externos de medias dos tipos imagens ou vídeos, e possuirá a relação tanto com Produto quanto com Fornecedor, ou seja, produtos e fornecedores possuirão imagens ou vídeos armazenadas na **mesma** tabela **media**.

```
php artisan make:model Media -msf
```

```
...tsi_daoo/20231/aula_05 <?php 8.2.5 main ✓
$ php artisan make:model Media -msf

INFO Model [app/Models/Media.php] created successfully.
INFO Factory [database/factories/MediaFactory.php] created successfully.
INFO Migration [database/migrations/2023_05_01_220932_create_media_table.php] created successfully.
INFO Seeder [database/seeder/MediaSeeder.php] created successfully.
```

Com os parâmetros **-msf** criaremos uma **migration**, um **seeder** e uma **factory** da classe **Media**. Vamos começar alterando a **migration** da model **Media**:

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('media', function (Blueprint $table) {
            $table->id();
            $table->bigInteger('model_id');
            $table->string('model_type');
            $table->string('type')->default('image'); //video
            $table->string('source');
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
```

```
{
    Schema::dropIfExists('media');
}
};
```

Os campos **model_id** e **model_type** serão utilizados para a funcionalidade polimórfica, ou seja, **model_id** guardará o ID da model Produto ou Fornecedor dependendo do campos **model_type**, o qual registrará o tipo de objeto Model a ser retornado.

Após a definição da **migration**, configuramos a model Media. Neste caso, não precisamos indicar o nome da tabela, pois o nome em inglês já está de acordo com a convenção, apenas adicionamos os campos **fillable** e indicamos o tipo de relacionamento polimórfico pelo método model:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\MorphTo;

class Media extends Model
{
    use HasFactory;
    protected $fillable = [
        'source',
        'type'
    ];

    public function model():MorphTo
    {
        return $this->morphTo();
    }
}
```

O método **model** retornará o relacionamento polimórfico, MorphTo, como usamos o nome do método igual aos prefixo dos campos **model_id** e **model_type**, não necessita passar o nome do campo como parâmetro.

Para a adicionar a relação nas classes **Produto** e **Fornecedor**, basta criar um método que retorne o relacionamento do tipo **MorphMany** e, como nas outras relações, repassar a classe de destino App\Models\Media como parâmetro, veja a seguir:

```
public function media():MorphMany
{
    return $this->morphMany(Media::class,'model');
}
```

No segundo parâmetro, passamos o prefixo utilizado nos campos responsáveis pelo polimorfismo (**model_id** e **model_type**).

Em `database\factories\MediaFactory.php`, iremos usar o helper fake para retornar uma URL de imagem aleatória, suando a api **via.placeholder.com**:

```
public function definition(): array
{
    return [
        'source' => fake()->imageUrl(360, 360, '', true),
    ];
}
```

Desta forma, ao criamos um factory de Media, mas apenas com a geração de imagens, para criar algumas relações com vídeos, iremos editar a seeder, `database\seeders\MediaSeeder.php`:

```
<?php

namespace Database\Seeders;

use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;
use App\Models\Media;
use App\Models\Produto;
use App\Models\Fornecedor;
use Carbon\Carbon;
use Exception;
use Illuminate\Support\Facades\Log;

class MediaSeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
```

```

public function run(): void
{
    try {

        $totalProduto = Produto::count();
        $totalFornecedor = Fornecedor::count();

        $createdAt = Carbon::now()→toDateTimeString();
        $modelProdutoid = fake()→numberBetween(1,$totalProduto);
        $modelFornecedorId = fake()→numberBetween(1,$totalFornecedor);
        $medias = [
            [
                'model_id' ⇒ $modelProdutoid,
                'model_type'⇒ 'App\Models\Produto',
                'source' ⇒ 'https://www.youtube.com/watch?v=QAfTYrDhdHE',
                'type' ⇒ 'video',
                'created_at'⇒ $createdAt,
                'updated_at'⇒ $createdAt
            ],
            [
                'model_id' ⇒ $modelProdutoid,
                'model_type'⇒ 'App\Models\Produto',
                'source' ⇒ 'https://www.youtube.com/watch?v=5s-_SnVI-1g',
                'type' ⇒ 'video',
                'created_at'⇒ $createdAt,
                'updated_at'⇒ $createdAt
            ],
            [
                'model_id' ⇒ $modelFornecedorId,
                'model_type'⇒ 'App\Models\Fornecedor',
                'source' ⇒ 'https://www.youtube.com/watch?v=127ng7botO4',
                'type' ⇒ 'video',
                'created_at'⇒ $createdAt,
                'updated_at'⇒ $createdAt
            ],
            [
                'model_id' ⇒ $modelFornecedorId,
                'model_type'⇒ 'App\Models\Fornecedor',
                'source' ⇒ 'https://www.youtube.com/watch?v=veSLRzmOfAI',
                'type' ⇒ 'video',
                'created_at'⇒ $createdAt,
                'updated_at'⇒ $createdAt
            ]
        ]
    }
}

```

```

        ],
        [
            'model_id' => $modelFornecedorId,
            'model_type' => 'App\Models\Fornecedor',
            'source' => 'https://www.youtube.com/watch?v=sBlkA95t1wM',
            'type' => 'video',
            'created_at' => $createdAt,
            'updated_at' => $createdAt
        ],
    ];

    Media::insert($medias)
        ? Log::channel('stderr')->info("Vídeos cadastrados! Fornecedor: $modelFornecedorId")
        : throw new Exception("Erro ao criar promoções!");

    } catch (Exception $error) {
        Log::debug($error->getMessage());
        Log::channel('stderr')->error($error->getMessage());
    }
}
}
}

```

Neste seeder sorteamos um produto e um fornecedor para cadastrar vídeos em cada um deles. Agora alteramos a classe **DatabaseSeeder**, adicionando o novo seeder de Media e adaptando as chamadas aos factories de Fornecedor e Produto:

```

<?php

namespace Database\Seeders;

// use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;
use App\Models\Fornecedor;
use App\Models\Produto;
use Illuminate\Support\Facades\Log;

class DatabaseSeeder extends Seeder
{
    /**
     * Seed the application's database.
     */
}

```

```

public function run(): void
{
    // \App\Models\User::factory(10)→create();

    // \App\Models\User::factory()→create([
    //     'name' ⇒ 'Test User',
    //     'email' ⇒ 'test@example.com',
    // ]);

    $this→call([
        RegiaoSeeder::class,
        EstadoSeeder::class,
        // PromocaoSeeder::class
    ]);

    //Sem PromocaoFactory
    // \App\Models\Fornecedor::factory(50)
    //     →has(Produto::factory()→hasAttached(
    //         Promocao::all()→random(2),
    //         [
    //             'created_at'⇒Carbon::now()→toDateTimeString(),
    //             'updated_at'⇒Carbon::now()→toDateTimeString(),
    //             'desconto'⇒50
    //         ],
    //         'promocoes'
    //     ))
    //     →create();

    //Com PromocaoFactory
    \App\Models\Fornecedor::factory(50)
        →has(Produto::factory(10)→hasAttached(
            Promocao::factory(3),
            [
                'created_at'⇒Carbon::now()→toDateTimeString(),
                'updated_at'⇒Carbon::now()→toDateTimeString(),
                'desconto'⇒35
            ],
            'promocoes'
        ))
        →create();
}

```

```

    $this->call([MediaSeeder::class]);
  }
}

```

No código acima, substituímos o método mágico **hasProdutos**, que chamava automaticamente a relação de Fornecedor com Produtos, pelo método padrão **has** passando a criação dos produtos manualmente através do método **factory** da model **Produto**. No método **factory** da classe **Produto** também chamamos o método mágico **hasMedia**, passando o parâmetro 3, ou seja, criaremos 25 Produtos com 3 imagens cada, para cada Fornecedor. Os fornecedores também possuirão pelo menos uma imagem, conforme determina a chamada ao método **hasMedia(1)**. Ao final, instanciamos o **MediaSeeder** e executamos o método **run()**.

Para testar, vamos recriar a base e executar novamente os seeders com o comando abaixo:

```
php artisan migrate:fresh --seed
```

Vamos usar o **tinker** para pesquisar por estes dois modelos que aparecem no final do log e carregar a sua relação com o modelo **Media**.

```

php artisan tinker
use App\Models\{Produto, Fornecedor, Media}
Produto::find(241)->load('media')
= App\Models\Produto {#4753

```

```

id: 241,
fornecedor_id: 10,
nome: "Hic ut eveniet deleniti ut.",
descricao: "Sit fugit iste quos fuga accusantium asperiores dolores neque molestiae debitis
preco: 4330.96,
qtd_estoque: 544,
importado: 1,
created_at: "2023-05-01 23:16:57",
updated_at: "2023-05-01 23:16:57",
media: Illuminate\Database\Eloquent\Collection {#4760
  all: [
    App\Models\Media {#4765
      id: 730,
      model_id: 241,
      model_type: "App\Models\Produto",
      type: "image",
      source: "https://via.placeholder.com/360x360.png/00eedd?text=+voluptate",
      created_at: "2023-05-01 23:16:57",
      updated_at: "2023-05-01 23:16:57",
    },
    App\Models\Media {#4768
      id: 731,
      model_id: 241,
      model_type: "App\Models\Produto",
      type: "image",
      source: "https://via.placeholder.com/360x360.png/0033cc?text=+voluptas",
      created_at: "2023-05-01 23:16:57",
      updated_at: "2023-05-01 23:16:57",
    },
    App\Models\Media {#4767
      id: 732,
      model_id: 241,
      model_type: "App\Models\Produto",
      type: "image",
      source: "https://via.placeholder.com/360x360.png/00dd11?text=+natus",
      created_at: "2023-05-01 23:16:57",
      updated_at: "2023-05-01 23:16:57",
    },
    App\Models\Media {#4770
      id: 761,
      model_id: 241,
      model_type: "App\Models\Produto",

```

```

    type: "video",
    source: "https://www.youtube.com/watch?v=QAfTYrDhdHE",
    created_at: "2023-05-01 23:16:58",
    updated_at: "2023-05-01 23:16:58",
  },
  App\Models\Media {#4771
    id: 762,
    model_id: 241,
    model_type: "App\Models\Produto",
    type: "video",
    source: "https://www.youtube.com/watch?v=5s-_SnVI-1g",
    created_at: "2023-05-01 23:16:58",
    updated_at: "2023-05-01 23:16:58",
  },
],
},
}

```

Conseguimos retornar o modelo de Produto com a coleção de objetos Media, contendo três imagens e dois vídeos.

No próximo exemplo vamos buscar o Fornecedor de id 9 e carregar sua relação com a tabela Media:

```

> Fornecedor::find(9)→load('media')
= App\Models\Fornecedor {#4777
  id: 9,
  nome: "Dias e Campos Ltda.",
  endereco: "Laudantium sequi optio. nr:2332",
  cnpj: "90.187.771/0001-22",
  telefone: "(63) 94073-1111",
  email: "padilha.ruth@caldeira.com.br",
  estado_id: 27,
  created_at: "2023-05-01 23:16:56",
  updated_at: "2023-05-01 23:16:56",
  media: Illuminate\Database\Eloquent\Collection {#4786
    all: [
      App\Models\Media {#4788
        id: 684,
        model_id: 9,
        model_type: "App\Models\Fornecedor",
        type: "image",

```

```

    source: "https://via.placeholder.com/360x360.png/009966?text=+sit",
    created_at: "2023-05-01 23:16:56",
    updated_at: "2023-05-01 23:16:56",
  },
  App\Models\Media {#4791
    id: 763,
    model_id: 9,
    model_type: "App\Models\Fornecedor",
    type: "video",
    source: "https://www.youtube.com/watch?v=127ng7botO4",
    created_at: "2023-05-01 23:16:58",
    updated_at: "2023-05-01 23:16:58",
  },
  App\Models\Media {#4790
    id: 764,
    model_id: 9,
    model_type: "App\Models\Fornecedor",
    type: "video",
    source: "https://www.youtube.com/watch?v=veSLRzmOfAI",
    created_at: "2023-05-01 23:16:58",
    updated_at: "2023-05-01 23:16:58",
  },
  App\Models\Media {#4793
    id: 765,
    model_id: 9,
    model_type: "App\Models\Fornecedor",
    type: "video",
    source: "https://www.youtube.com/watch?v=sBIkA95t1wM",
    created_at: "2023-05-01 23:16:58",
    updated_at: "2023-05-01 23:16:58",
  },
],
},
}

```

Novamente, rebemos os dados do modelo Fornecedor com a coleção de modelos Media, contendo, neste caso, uma imagem e três vídeos.

Agora vamos fazer o caminho inverso, buscar uma media específica e carregar a sua relação com modelos Produto ou Fornecedor:

Primeiro pesquisaremos pela media de ID=730, deverá retornar o Produto de ID=241, em seu atributo model:

```

> Media::find(730)→load('model')
= App\Models\Media {#4781
  id: 730,
  model_id: 241,
  model_type: "App\Models\Produto",
  type: "image",
  source: "https://via.placeholder.com/360×360.png/00eedd?text=+voluptate",
  created_at: "2023-05-01 23:16:57",
  updated_at: "2023-05-01 23:16:57",
  model: App\Models\Produto {#4803
    id: 241,
    fornecedor_id: 10,
    nome: "Hic ut eveniet deleniti ut.",
    descricao: "Sit fugit iste quos fuga accusantium asperiores dolores neque molestiae debiti",
    preco: 4330.96,
    qtd_estoque: 544,
    importado: 1,
    created_at: "2023-05-01 23:16:57",
    updated_at: "2023-05-01 23:16:57",
  },
}

```

Observe que o atributo model é na verdade um objeto do tipo Produto.

Vejamos mais um exemplo com uma media do Fornecedor:

```

> Media::find(763)→load('model')
= App\Models\Media {#4810
  id: 763,
  model_id: 9,
  model_type: "App\Models\Fornecedor",
  type: "video",
  source: "https://www.youtube.com/watch?v=127ng7botO4",
  created_at: "2023-05-01 23:16:58",
  updated_at: "2023-05-01 23:16:58",
  model: App\Models\Fornecedor {#4825
    id: 9,
    nome: "Dias e Campos Ltda.",
    endereco: "Laudantium sequi optio. nr:2332",
    cnpj: "90.187.771/0001-22",
    telefone: "(63) 94073-1111",
    email: "padilha.ruth@caldeira.com.br",
  },
}

```

```

        estado_id: 27,
        created_at: "2023-05-01 23:16:56",
        updated_at: "2023-05-01 23:16:56",
    },
}

```

Pesquisando pela Media de ID=763, o objeto do atributo **model** retornado agora é o Fornecedor de ID=9.

SQL DUMP E LOGGING

Outra forma de acessar um dump do comando SQL toda vez que for executada uma query no banco, é através da utilização do método **listen** da classe **DB** e da configuração um log personalizado (**query**) iniciado na **AppServiceProvider**:

```

<?php

namespace App\Providers;

use Illuminate\Database\Events\QueryExecuted;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Log;
use Illuminate\Support\ServiceProvider;

class AppServiceProvider extends ServiceProvider
{
    /**
     * Register any application services.
     */
    public function register(): void
    {
        //
    }

    /**
     * Bootstrap any application services.
     */
    public function boot(): void
    {
        DB::listen(function (QueryExecuted $query) {
            $dumpQuery = [

```

```

        'SQL' => $query->sql,
        'BINDINGS' => $query->bindings,
        'TIME(ms)' => $query->time,
    ];
    Log::channel('query')->info(print_r($dumpQuery,true));
    // Log::debug(print_r($dumpQuery,true));
    });
}
}

```

Configurar o canal **"query"** em **config/logging.php** adicionando o novo item no array **channel**:

```

...
'query' => [
    'driver' => 'single',
    'level' => env('LOG_LEVEL', 'debug'),
    'path' => storage_path('logs/query.log'),
],
];

```

Agora toda vez que executarmos uma query no banco, pode ser pelo **tinker** ou uma chamada a algum **controller**, ou seja, em qualquer parte do sistema, o arquivo **query.log** será atualizado com o registro do SQL. Podemos ainda monitorá-lo com os seguintes comandos em uma outra aba do terminal:

Bash (linux/windos com git-bash)

Visualizar no terminal com o seguinte comando:

```
clear;echo "Reset:\n" > storage/logs/query.log; tail -f storage/logs/query.log
```

Onde:

- **clear** : limpa o terminal
- **echo "Reset:\n" >** : apaga o conteúdo do último log escrevendo a palavra **"Reset:"** com uma nova linha no arquivo de saída de log em **storage/logs/query.log**
- **tail -f storage/logs/query.log** : imprimir no terminal o final do arquivo assim como cada nova escrita no mesmo.

Powershell

Comando equivalente ao **bash** com o **powershell**

```
clear;echo "Log Reseted:\n" > ".\storage\logs\query.log"; Get-Content -Path ".\storage\logs\qu
```

Onde:

- `clear` : limpa o terminal
- `echo "Reset:\n" >` : apaga o conteúdo do último log escrevendo a palavra **"Reset:"** com um nova linha no arquivo de saída de log em **storage/logsl/query.log**
- `Get-Content -Path storage/logs/query.log -Wait` : imprimir no terminal o final do arquivo, assim como, toda nova escrita no mesmo.

UTILIZANDO O DEBUGBAR:

Outra forma, bem mais conveniente de logar os SQLs e ainda outras características da requisição, ou seja, debugar as chamadas ao Laravel, é através do **PHP Debugbar** adaptado para o framework pelo pacote a seguir.

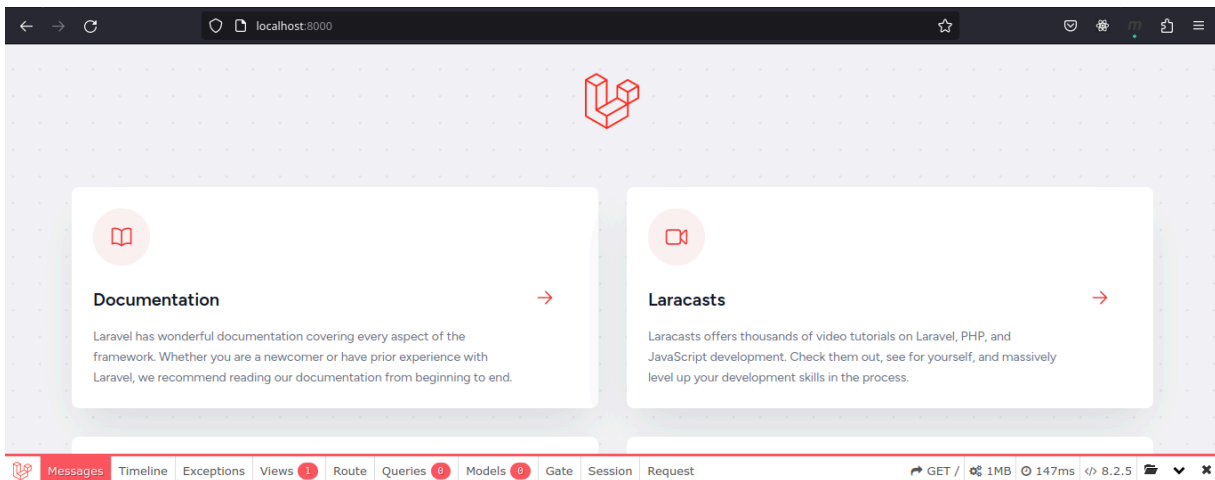
```
https://github.com/barryvdh/laravel-debugbar
```

Instalação:

```
composer require barryvdh/laravel-debugbar --dev
```

Nesta instalação usamos o `--dev` pois é uma dependência de desenvolvimento, não é necessário em produção.

Agora ao acessar a aplicação web veremos uma barra na parte inferior com logo do laravel, como mostra a figura abaixo:



O **Debugbar** estará sempre ativado de acordo com a variável de ambiente `APP_DEBUG`, ou seja, quando a mesma estiver `true`, a barra estará ativa.

Para organizar melhor a nossa aplicação, vamos criar uma pasta para guardar apenas rotas específicas para debugar, portanto crie a estrutura da figura abaixo em `routes→debug→queries.php`:



E no final do arquivo `routes/web.php` criaremos uma rota com o prefixo **debug** que deverá funcionar apenas quando a variável de ambiente **APP_DEBUG** estiver ativa. Portanto adicione o seguinte código ao final do arquivo de configuração das rotas web.

```
Route::prefix('debug')→group(function(){

    Route::get('/',function(){
        if(!env('APP_DEBUG'))
            return redirect()→route('welcome');
    });
    if(env('APP_DEBUG'))
        require __DIR__.'/debug/queries.php';
});
```

É preciso dar um nome a rota da página de welcome, usando o método `name`:

```
Route::get('/', function () {
    return view('welcome');
})->name('welcome');
```

Toda vez que a rota **debug** for acessada mas a **APP_DEBUG** for false, em produção por exemplo, a requisição será redirecionada a página de welcome.

Agora criaremos as rotas de debug no arquivo debug/queries.php, conforme o código abaixo:

```
<?php

use App\Models\{Regiao, Estado, Fornecedor, Produto, Promocao};
use Illuminate\Support\Facades\Route;
use Barryvdh\Debugbar\Facades\Debugbar;

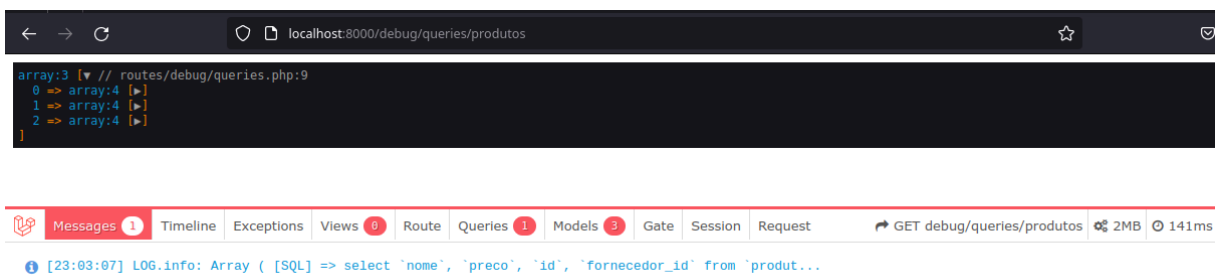
Route::prefix('queries')->group(function () {
    Route::get('produtos/', function () {
        dump(
            Produto::whereBetween('preco', [100,500])
                ->where('importado', true)
                ->get(['nome','preco','id','fornecedor_id'])->toArray()
        );
    });

    Route::get('produtos-fornecedor/', function () {
        dump(
            Produto::with('fornecedor')
                ->get(['nome','preco','id','fornecedor_id'])
                ->toArray()
        );
    });

    Route::get('fornecedores-estados/',function(){
        $listEstados = collect([]);
        Fornecedor::all()->each(function ($f) use ($listEstados){
            $listEstados->push($f->estado->uf);
        });
        Debugbar::error($listEstados);
        dump($listEstados);
    });
});
```

```
Route::get('fornecedores-estados-load/',function(){
    $listEstados = collect([]);
    Fornecedor::all()→load('estado')→each(function ($f) use ($listEstados){
        $listEstados→push($f→estado→uf);
    });
    Debugbar::error($listEstados);
    dump($listEstados);
});
});
```

Acessando a rota **debug/queries/produtos**:



Referências:

- Relações Polimórficas
 - <https://laravel.com/docs/10.x/eloquent-relationships#polymorphic-relationships>
 - <https://moedayraki.medium.com/laravel-push-method-for-saving-models-and-relationships-f5936596acbe>
- Relação Recursiva
 - <https://medium.com/@shaunthornburgh/using-parent-child-recursive-relationships-in-laravel-883261de6ff6>
- Debugging Queries:
 - <https://write.corbpie.com/how-to-debug-and-view-a-laravel-db-query/>
 - <https://laraveldaily.com/post/laravel-eloquent-tools-debug-slow-sql-queries>
 - <https://www.youtube.com/watch?v=SR3RzlfeozI>
 - <https://github.com/barryvdh/laravel-debugbar/blob/master/readme.md>
 - <https://laravel.com/docs/9.x/telescope>

- <https://medium.com/@alirazalilani/debugging-php-laravel-with-visual-studio-code-37b756fb6c19>

LICENÇA:

Creative Commons — Attribution-ShareAlike 4.0 International
— CC BY-SA 4.0